

Technical Requirements Document

Vibe Code Security Scanner

Version: 1.0

Author: Jude Perera

Project: Vibe Code Security Scanner

Date: August 2026

1. Tech Stack

Component	Technology	Version	Rationale
Language	Python	3.11+	Native AST module for code analysis, wide security library support
Web Framework	FastAPI	0.100+	Async support, automatic OpenAPI docs, Pydantic validation
ASGI Server	Uvicorn	Latest	Production-grade ASGI server for FastAPI
Database	SQLite	Built-in	Zero config, sufficient for single-instance deployment, no external dependency
ORM / DB Layer	sqlite3 (stdlib)	Built-in	Lightweight, no ORM overhead needed for this use case
Data Validation	Pydantic	2.x	Request body validation via FastAPI integration
Static Analysis	Python ast (stdlib)	Built-in	Parse Python source into AST for precise, low false-positive checks
Pattern Matching	re (stdlib)	Built-in	Regex-based checks for frontend files and string patterns
Frontend	HTML / CSS / JS	Vanilla	No framework dependency, zero build step, fast load
Testing	pytest	Latest	Industry standard, simple fixture model, good CI integration
CI/CD	GitHub Actions	Latest	Native GitHub integration, free for public repos
SAST	Bandit	Latest	Python-specific security linting
SAST	Semgrep	Latest	OWASP Top 10 ruleset, multi-language
Dependency Audit	Safety	Latest	CVE database check for Python dependencies
Secret Scanning	Gitleaks	Latest	Full git history scanning for accidentally committed secrets

2. Database Design

2.1 Schema

Table: scans

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique scan identifier
repo_url	TEXT	NOT NULL	GitHub repository URL that was scanned
timestamp	TEXT	NOT NULL	ISO 8601 UTC timestamp of scan completion
total_findings	INTEGER	NOT NULL	Total number of findings across all checks
vulnerability_rate	REAL	NOT NULL	Findings per scanned Python file (percentage)

Table: findings

Column	Type	Constraints	Description
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique finding identifier
scan_id	INTEGER	FOREIGN KEY → scans.id	Parent scan this finding belongs to
check_name	TEXT	NOT NULL	Name of the security check that fired
severity	TEXT	NOT NULL	HIGH, MEDIUM, or LOW
file_path	TEXT	NOT NULL	Relative path to the affected file
line_number	INTEGER	NOT NULL	Line number of the finding
detail	TEXT	NOT NULL	Plain English explanation of the finding

2.2 Database Decisions

- SQLite chosen over PostgreSQL — no external database server required, sufficient for single-instance deployment, zero configuration
- `row_factory = sqlite3.Row` used so query results return as dictionaries, not tuples
- Parameterised queries used throughout — no string formatting in SQL
- Foreign key constraint enforces referential integrity between findings and scans
- No ORM used — raw `sqlite3` keeps the dependency count minimal

3. Authentication

3.1 Current State

The application has no authentication. It is a public tool designed to scan public GitHub repositories. Any user can access the dashboard and submit a scan.

3.2 Security Mitigations Without Auth

- Cloned repositories deleted immediately after scanning — no persistent attack surface
- Only public GitHub URLs accepted — no credential handling
- No sensitive user data stored — only repo URLs and finding details
- Git clone timeout prevents hanging on slow repos
- Temp directories use secure random names via `tempfile.mkdtemp()`

3.3 Future Authentication Roadmap

- JWT-based authentication for user accounts
 - Rate limiting per IP using `slowapi`
 - Private repository scanning via GitHub OAuth
 - API key authentication for programmatic access
-

4. Security Architecture

4.1 Scanning Isolation

- Each scan runs in an isolated temporary directory
- Directory path: `/tmp/scan_<random>/repo/`
- Directory deleted in a finally block — guaranteed cleanup on success and failure
- No inter-scan contamination possible

4.2 Input Validation

- `repo_url` validated as non-empty string by Pydantic model
- git clone exit code checked — non-zero returns 400 error
- Exception handling wraps the entire scan process
- HTTP 400 returned for all scan failures with a user-readable message

4.3 CI/CD Security

Job	Tool	What it checks
bandit	Bandit	Python code for insecure functions, hardcoded secrets, weak crypto
semgrep	Semgrep	OWASP Top 10 rules, secrets patterns across codebase
dependency-audit	Safety	Python dependencies against CVE database
secret-scan	Gitleaks	Full git history for accidentally committed credentials
unit-tests	pytest	All 6 security check unit tests — pipeline fails if any fail

5. File Structure

Path	Purpose
<code>app/main.py</code>	FastAPI application, route definitions, scan orchestration

Path	Purpose
app/database.py	SQLite connection, table creation, CRUD functions
app/analyser.py	Directory walker, coordinates check execution by file type
app/checks/python_checks.py	14 AST-based Python security check functions
app/checks/frontend_checks.py	2 regex-based HTML/JS security check functions
static/index.html	Single-page web dashboard — HTML, CSS, JS in one file
tests/test_analyser.py	6 unit tests covering core check functions
.github/workflows/security-pipeline.yml	5-job CI/CD security pipeline
requirements.txt	Python dependencies
.gitignore	Excludes .db, .env, .venv, __pycache__, *.json

6. Key Technical Decisions

6.1 AST vs Regex for Python Analysis

Python files are analysed using the `ast` module rather than `regex`. AST parsing converts source code into a syntax tree, allowing checks to understand code structure — function definitions, decorators, arguments, return values — rather than pattern matching text. This produces significantly fewer false positives than `regex`-based Python analysis.

6.2 Synchronous vs Async Scanning

Scanning runs synchronously within a FastAPI route handler. This is acceptable for the current use case — single-user tool, scans complete in under 60 seconds. A production version handling concurrent users would move scanning to a background task queue using Celery or FastAPI `BackgroundTasks`.

6.3 SQLite vs PostgreSQL

SQLite was chosen for zero-configuration deployment. The tool is designed as a single-instance application. PostgreSQL would be required for multi-instance deployment, concurrent write performance, or user account management — none of which are current requirements.

6.4 Vanilla JS vs React

The frontend is vanilla HTML, CSS, and JavaScript with no framework. This eliminates a build step, reduces complexity, and keeps the project focused on the security engineering content rather than frontend engineering. The dashboard is simple enough that a framework would add overhead without value.